Training Giants

AdaFactor, LAMB, LARS & NovoGrad

Lesson 6 of the Optimizer Course

What changes when the model has a billion weights
and the optimizer runs out of memory

Everything so far worked on a two-weight bowl. Real models have billions of weights, and at that scale two new enemies appear that a toy can't show: the optimizer literally runs out of memory, and huge training batches destabilise. This lesson is about the optimizers built to survive scale — so we'll swap the toy bowl for honest back-of-the-envelope numbers where the problem actually lives.

Marevlo Research

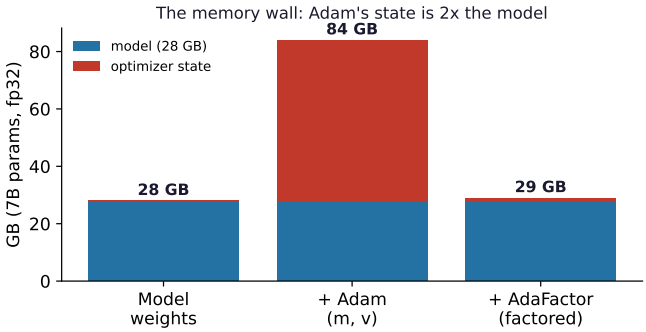
1 Two problems that only appear at scale

Adam is wonderful, but look at what it stores: for *every* weight it keeps a momentum value m *and* a scale value v. That's **two extra numbers per weight** — so the optimizer's memory is *twice the size of the model itself.*

Let's actually do the numbers

A 7-billion-parameter model, stored in 32-bit floats (4 bytes each):

- Model weights: $7\text{B} \times 4 = 28$ GB.
- Adam's m and v: another $2 \times 28 = 56$ GB.
- **Total just to train: 84 GB** — and the optimizer state (56 GB) is bigger than the model. That often doesn't fit on a single GPU.



There's a second, separate problem. To train fast on hundreds of GPUs you use **huge batches** (tens of thousands of examples per step). Big batches give cleaner gradients, which should let you use a big learning rate — but a single global learning rate destabilises, because different *layers* have gradients of wildly different sizes. The early layers and late layers need totally different step sizes.

The big idea

At scale, two new bottlenecks dominate: **memory** (the optimizer state is huge) and **batch size** (one global learning rate can't serve all layers). The four optimizers in this lesson each attack one of these. None of them is something you'd reach for on a small model — they're scale tools.

2 AdaFactor: shrink the memory by factoring

AdaFactor (used to train Google's T5) attacks the memory problem with a neat linear-algebra trick. Adam's scale value v for a weight *matrix* of shape $n \times m$ is itself an $n \times m$ grid of numbers. AdaFactor refuses to store the whole grid. Instead it stores just the **row sums** (n numbers) and **column sums** (m numbers), and *reconstructs* an approximation of any entry as (its row sum $\times$ its column sum) $\div$ the grand total.

Let's actually do the numbers

Suppose a weight matrix's squared-gradient grid is

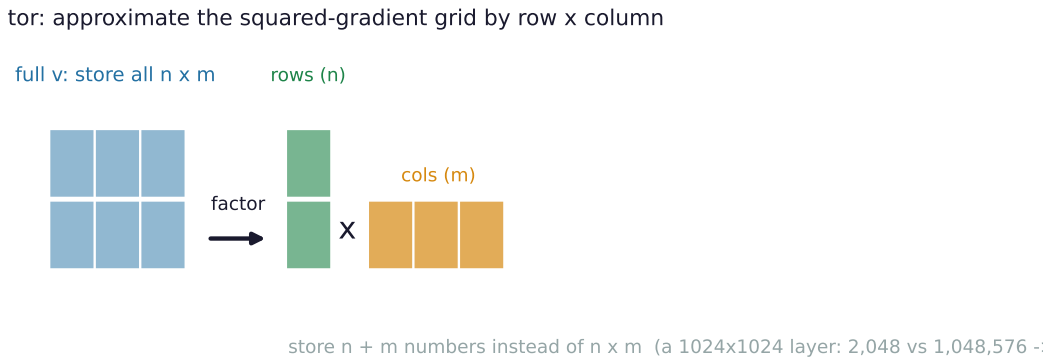
$$g^2 = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 6 & 4 \end{bmatrix}.$$

Row sums $R = (6, 12)$; column sums $C = (3, 9, 6)$; total $S = 18$. Reconstruct entry (i, j) as $R_i C_j / S$:

$$\hat{v}_{1,1} = \tfrac{6 \cdot 3}{18} = 1, \quad \hat{v}_{1,2} = \tfrac{6 \cdot 9}{18} = 3, \quad \hat{v}_{2,2} = \tfrac{12 \cdot 9}{18} = 6, \ \ldots$$

The reconstruction is **exact** here, because this grid happens to be a clean "row pattern × column pattern" (rank-1). Store R and C: that's $2 + 3 = 5$ numbers instead of 6.

Real grids aren't perfectly rank-1, so it's an *approximation* — bump the $(2, 2)$ entry from 6 to 8 and the reconstruction gives 7.7 instead of 8. Close enough in practice. And the memory win is enormous: for a 1024×1024 layer, $n + m = 2{,}048$ numbers replace $n \times m = 1{,}048{,}576$ — a **512× reduction.**



AdaFactor also typically drops the momentum m entirely (saving the other half of the memory). The result: you can train an 11-billion-parameter model where plain Adam's state simply wouldn't fit.

3 LARS: a learning rate per layer

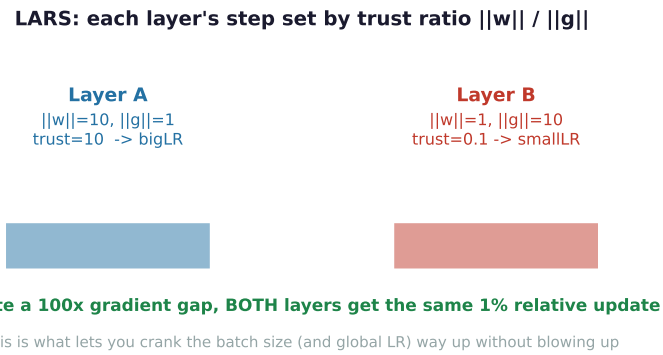
LARS attacks the *big-batch* problem. The core observation: what matters for stability isn't the absolute step size, it's the step size *relative to the weights*. A layer whose weights have size 10 can absorb a much bigger step than a layer whose weights have size 1. So LARS sets each layer's learning rate using its **trust ratio** $= \dfrac{\|w\|}{\|g\|}$.

Let's actually do the numbers

Global LR $\eta = 0.01$. Two layers with very different gradient scales:

- Layer A: $\|w\| = 10$, $\|g\| = 1$. Trust $= 10$, so effective LR $= 0.01 \times 10 = 0.1$.
- Layer B: $\|w\| = 1$, $\|g\| = 10$. Trust $= 0.1$, so effective LR $= 0.01 \times 0.1 = 0.001$.

Now check the *relative* update each layer actually makes, $\frac{\|\Delta w\|}{\|w\|} = \text{effLR} \times \frac{\|g\|}{\|w\|}$: for A it's $0.1 \times \frac{1}{10} = 0.01$; for B it's $0.001 \times \frac{10}{1} = 0.01$. **Both layers move by exactly 1% of their size**, despite a 100× gap in gradient magnitude. That uniformity is what lets you crank the batch size (and global LR) way up without any layer blowing up.



LARS was the key to large-batch **vision** training — ResNet on huge batches, and the self-supervised SimCLR.

4 LAMB and NovoGrad: the same ideas, remixed

LAMB is simply "LARS applied on top of Adam." Take Adam's update direction, then rescale it per layer by the trust ratio. This combination let researchers train **BERT in 76 minutes** (down from 3 days) by pushing the batch size into the tens of thousands. If you're doing large-batch transformer pretraining, LAMB is the name you'll meet.

NovoGrad attacks memory from a different angle than AdaFactor: instead of one scale value v per *weight*, it keeps one v per *layer* — a single number summarising that whole layer's gradient size. That's almost no optimizer memory at all, plus layer-wise normalisation for stability. It became popular for **speech** models (NVIDIA's Jasper / QuartzNet), which stack many layers and benefit from the low footprint.

When you're actually training

You will not use any of these on a normal-sized model — the default stays **AdamW**. Reach for them only when you hit a scale wall: **AdaFactor** (or 8-bit Adam, a related trick) when you're *memory*-bound; **LAMB** when you're scaling *batch size* on transformers; **LARS** for large-batch vision; **NovoGrad** for deep speech stacks on a tight memory budget. Knowing which wall you've hit tells you which tool to grab.

5 What you can do now, and what's next

You can now explain why Adam's memory is 2× the model, how AdaFactor's row×column factoring shrinks it, why big-batch training needs per-layer learning rates, and what the trust ratio does — plus the one-line role of LAMB and NovoGrad.

Next — Lesson 7: Using Curvature. Step back and notice something: *every* optimizer from Lessons 2–6 used only the gradient — the slope. They *guessed* at curvature cheaply (a diagonal estimate, a factored one), but never used the real thing. In Lesson 1 we met the Hessian, the true curvature. What if we actually used it? You get **Newton's method**, which on our bowl reaches the exact bottom in a *single step* — and the whole story of why we can't just always do that.